



MODULE 1 – DART ESSENTIEL

Formation Flutter Professionnelle – 50 heures

SUPPORT DE COURS

1. Introduction à Dart

Dart est le langage de programmation développé par Google, optimisé pour le développement d'interfaces utilisateur. Il se compile en :

- **JIT (Just In Time)** : pendant le développement, pour le hot reload ultra-rapide.
- **AOT (Ahead Of Time)** : pour la production, générant un code natif performant.

Pourquoi Dart pour Flutter ?

- Syntaxe familière (proche de Java, C#, JavaScript)
- Null safety intégré nativement (depuis Dart 2.12)
- Programmation orientée objet complète
- Asynchrone performant (Future, Stream, Isolates)

2. Variables, Types et Null Safety

Déclaration de variables

```
// Type explicite
String nom = 'Alice';
int age = 30;
double prix = 19.99;
bool actif = true;

// Inférence de type (var)
var ville = 'Paris'; // Dart infère String

// final : valeur assignée une seule fois, déterminée à l'exécution
final heureActuelle = DateTime.now();

// const : valeur constante à la compilation
const pi = 3.14159;
const listeFixe = [1, 2, 3]; // Immutable
```

Null Safety (Sécurité des null) Dart interdit les valeurs null par défaut. C'est une révolution pour la fiabilité.

```
// ❌ Erreur : une variable non nullable ne peut pas être null
String nom;
nom = null; // Compile error
```



```
//  Correct : type nullable avec ?
String? nom;
nom = null; // OK

// Opérateurs null safety
String? prenom;
print(prenom ?? 'Inconnu');           // ?? : valeur par défaut si null
print(prenom?.toUpperCase());        // ?. : appel sécurisé si non null
prenom ??= 'Invité';                  // ??= : assigne seulement si null
String nomCourt = prenom;             // ! : assertion (je garantis que ce n'est pas null)
```

Règle d'or : utilisez ? uniquement quand la valeur null a un sens métier. Sinon, gardez le type non nullable.

3. Fonctions

```
// Fonction simple
int addition(int a, int b) {
    return a + b;
}

// Arrow function (fonction fléchée)
int multiplication(int a, int b) => a * b;

// Paramètres nommés (recommandé pour la clarté)
void creerUtilisateur({required String nom, required int age, String? email}) {
    print('$nom, $age ans, email: ${email ?? "non renseigné"}');
}

// Appel
creerUtilisateur(nom: 'Bob', age: 25);

// Paramètres positionnels optionnels
void afficher([String message = 'Bonjour']) {
    print(message);
}
```

Bonnes pratiques Flutter : préférez les paramètres nommés pour les widgets et fonctions complexes. Cela améliore la lisibilité du code.

4. Structures de contrôle

```
// Conditions
int score = 85;
```



```
if (score >= 90) {  
    print('A');  
} else if (score >= 80) {  
    print('B');  
} else {  
    print('C');  
}  
  
// Switch expression (Dart 3)  
var grade = switch (score) {  
    >= 90 => 'A',  
    >= 80 => 'B',  
    >= 70 => 'C',  
    _ => 'F',  
};  
  
// Boucles  
for (int i = 0; i < 5; i++) {  
    print(i);  
}  
  
// for-in  
List<String> fruits = ['Pomme', 'Banane', 'Orange'];  
for (var fruit in fruits) {  
    print(fruit);  
}  
  
// while / do-while  
int compteur = 0;  
while (compteur < 3) {  
    print(compteur);  
    compteur++;  
}
```

5. Collections

List (liste ordonnée, indexée)

```
List<int> nombres = [1, 2, 3, 4, 5];  
  
// Méthodes fonctionnelles  
  
var pairs = nombres.where((n) => n.isEven).toList();           // [2, 4]  
var doubles = nombres.map((n) => n * 2).toList();               // [2, 4, 6, 8, 10]
```



```
var somme = nombres.fold(0, (acc, n) => acc + n); // 15
var premier = nombres.firstWhere((n) => n > 3); // 4

// Spread operator

List<int> plusGrands = [...nombres, 6, 7, 8];
```

Map (clé-valeur)

```
Map<String, dynamic> utilisateur = {
  'nom': 'Alice',
  'age': 30,
  'actif': true,
};

print(utilisateur['nom']); // Alice
utilisateur['email'] = 'alice@mail.com';

// Itération
utilisateur.forEach((cle, valeur) {
  print('$cle : $valeur');
});
```

Set (ensemble, valeurs uniques)

```
Set<String> tags = {'flutter', 'dart', 'mobile'};
tags.add('flutter'); // Ignoré (déjà présent)
```

6. Programmation Orientée Objet

Classes et constructeurs

```
class Utilisateur {
  final String id;
  final String nom;
  final int age;
  final String? email;

  // Constructeur principal
  Utilisateur({
    required this.id,
    required this.nom,
    required this.age,
    this.email,
  });

  // Named constructor (constructeur nommé)
```



```
Utilisateur.anonyme({required this.id})  
    : nom = 'Anonyme',  
      age = 0,  
      email = null;  
  
// Méthode  
String sePresenter() => 'Je suis $nom, $age ans.';  
  
// toString  
@override  
String toString() => 'Utilisateur(id: $id, nom: $nom);'  
}
```

Héritage

```
class Employe extends Utilisateur {  
    final String departement;  
    final double salaire;  
  
    Employe({  
        required super.id,  
        required super.nom,  
        required super.age,  
        required this.departement,  
        required this.salaire,  
        super.email,  
    });  
  
    @override  
    String sePresenter() {  
        return '${super.sePresenter()} Je travaille à $departement.';  
    }  
}
```

Mixins (comportements réutilisables)

```
mixin Loggable {  
    void log(String message) {  
        print('[LOG ${DateTime.now()}] $message');  
    }  
}  
  
class Service with Loggable {  
    void executer() {  
        log('Service en cours d\'exécution');  
    }  
}
```



Quand utiliser quoi ?

- extends : relation “est un” (Employe est un Utilisateur)
- implements : contrat d’interface (doit redéfinir tout)
- with : réutilisation de comportements (Loggable, Serializable)

7. Programmation Asynchrone

Future et async/await

```
// Fonction asynchrone simulant un appel réseau
Future<String> recupererNomUtilisateur(String id) async {
  await Future.delayed(Duration(seconds: 2)); // Simulation latence
  return 'Utilisateur $id';
}

// Utilisation
void main() async {
  print('Chargement...');
  final nom = await recupererNomUtilisateur('123');
  print(nom); // Après 2 secondes
}
```

Gestion des erreurs

```
Future<void> chargerDonnees() async {
  try {
    final data = await fetchData();
    print(data);
  } catch (e) {
    print('Erreur : $e');
  } finally {
    print('Terminé');
  }
}
```

Future.wait (parallélisme)

```
final results = await Future.wait([
  fetchUtilisateurs(),
  fetchCommandes(),
  fetchProduits(),
]);
```

En Flutter, tout ce qui touche au réseau, au stockage, ou à l’UI est asynchrone. Maîtriser async/await est **indispensable** avant d’écrire une seule ligne de widget.



ÉNONCÉS DES TRAVAUX PRATIQUES

TP1 – Gestionnaire de Tâches en Console (45 min)

Contexte Vous développez une application de gestion de tâches en ligne de commande. C'est le premier bloc de votre fil rouge : le modèle de données et la logique métier.

Objectifs pédagogiques

- Modéliser une classe avec null safety
- Manipuler une List avec les méthodes fonctionnelles
- Organiser le code en classes séparées

Consignes **Étape 1 – Modéliser la tâche** Créez une classe Tache avec les propriétés suivantes :

- id (String, final, requis)
- titre (String, requis)
- description (String?, nullable)
- estTerminee (bool, default false)
- dateCreation (DateTime, final, requis)
- priorite (enum Priorite : basse, moyenne, haute)

Ajoutez :

- Un constructeur avec paramètres nommés (required où nécessaire)
- Une méthode toJson() qui retourne un Map<String, dynamic>
- Une méthode fromJson(Map<String, dynamic> json) factory
- Un toString() formaté : [HAUTE] Acheter du lait - En cours

Étape 2 – Créer le gestionnaire Créez une classe GestionnaireTaches qui contient une List<Tache>.

Implémentez les méthodes :

- ajouter(Tache tache) : ajoute une tâche
- supprimer(String id) : supprime par ID
- marquerCommeTerminee(String id) : toggle le booléen
- listerToutes() : retourne toutes les tâches (triées par date décroissante)
- listerParPriorite(Priorite p) : filtre par priorité
- listerTerminees() : filtre les tâches terminées
- compterEnCours() : retourne le nombre de tâches non terminées
- rechercher(String terme) : recherche dans le titre et la description (insensible à la casse)

Étape 3 – Menu interactif en console Créez une fonction main() qui :

1. Instancie le gestionnaire



2. Pré-remplit 5 tâches de démonstration

3. Affiche un menu : `=== GESTIONNAIRE DE TÂCHES ===`

1. Ajouter une tâche	2. Lister toutes	3. Lister par
priorité	4. Marquer comme terminée	5. Supprimer
6. Rechercher	7. Statistiques	8.
Quitter		

4. Boucle jusqu'à ce que l'utilisateur choisisse 8

Livrable attendu : Un fichier `main.dart` exécutable en console.

TP2 – Manipulation Fonctionnelle de Collections (30 min)

Contexte Vous recevez une liste brute de données utilisateurs et devez les transformer pour un tableau de bord.

Données de départ

```
final utilisateurs = [
  {'nom': 'Alice', 'age': 30, 'ville': 'Paris', 'actif': true},
  {'nom': 'Bob', 'age': 17, 'ville': 'Lyon', 'actif': false},
  {'nom': 'Charlie', 'age': 25, 'ville': 'Paris', 'actif': true},
  {'nom': 'Diana', 'age': 42, 'ville': 'Marseille', 'actif': true},
  {'nom': 'Evan', 'age': 19, 'ville': 'Lyon', 'actif': false},
];
```

Consignes Réalisez les opérations suivantes **uniquement avec les méthodes fonctionnelles** (where, map, fold, any, every, groupBy si vous importez collection) :

1. **Filtrer** : liste des utilisateurs actifs majeurs (≥ 18 ans)
2. **Transformer** : liste des noms en majuscules des utilisateurs parisiens
3. **Agréger** : âge moyen de tous les utilisateurs actifs (utilisez fold)
4. **Vérifier** : y a-t-il au moins un utilisateur de plus de 40 ans ? (any)
5. **Grouper** : créez une `Map<String, List<String>>` où la clé est la ville et la valeur est la liste des noms
6. **Trier** : liste des utilisateurs triés par âge décroissant

Livrable attendu : Fonction `analyserUtilisateurs()` qui affiche chaque résultat avec `print()`.

TP3 – Simulation Asynchrone et JSON (30 min)

Contexte Vous simulez l'appel à une API distante qui retourne des tâches au format JSON. Vous devez parser ces données et les intégrer dans votre gestionnaire.

Consignes Étape 1 – Simuler le serveur Créez une fonction `Future<List<Map<String, dynamic>>>` `fetchTachesDepuisServeur()` qui :

- Attend 2 secondes (`Future.delayed`)
- Retourne une liste de 3 tâches au format JSON : dart `[` `{'id': '101', 'titre': 'Apprendre Flutter', 'description': null, 'estTerminee': false, 'dateCreation': '202608-01T10:00:00Z', 'priorite': 'haute'},` `{'id': '102', 'titre': 'Maîtriser Dart', 'description': 'Null safety et asynchrone', 'estTerminee': true, 'dateCreation': '2026-0728T14:30:00Z', 'priorite': 'moyenne'},` `{'id': '103', 'titre': 'Déployer sur les stores', 'description': null, 'estTerminee': false, 'dateCreation': '2026-08-03T09:15:00Z', 'priorite': 'basse'},` `]`



Étape 2 – Parser le JSON Créez une fonction Tache `tacheFromJson(Map<String, dynamic> json)` qui :

- Parse les champs
- Convertit la chaîne `dateCreation` en `DateTime.parse()`
- Convertit la chaîne `priorite` en `enum Priorite`
- Gère le `description` nullable

Étape 3 – Intégration asynchrone Dans votre `main()`, créez un flux asynchrone qui :

1. Affiche “Connexion au serveur...”
2. Récupère les tâches avec `await`
3. Les convertit en objets Tache
4. Les injecte dans votre `GestionnaireTaches`
5. Affiche “3 tâches chargées avec succès”
6. Liste les tâches non terminées

Étape 4 – Gestion d’erreur Simulez une erreur (1 fois sur 2 avec `Random`) dans `fetchTachesDepuisServeur`. Attrapez l’exception dans le `main()` et affichez un message utilisateur compréhensible.

Livrable attendu : Fichier `main_async.dart` démontrant le flux complet.

FICHE RÉCAP STAGIAIRE – Dart Essentiel

Points clés à retenir

Concept

Syntaxe / Règle

Variable mutable	<code>var nom = 'Alice';</code>
Variable immuable	<code>final nom = 'Alice';</code> ou <code>const nom = 'Alice';</code>
Nullable	<code>String? nom;</code> (peut être null)

Concept	Syntaxe / Règle
Non-nullable	<code>String nom;</code> (doit être initialisé)
Valeur par défaut	<code>nom ?? 'Inconnu'</code>
Appel sécurisé	<code>nom?.toUpperCase()</code>
Fonction fléchée	<code>int carre(int x) => x * x;</code>
Paramètres nommés	<code>{required String nom, int? age}</code>
List immutable	<code>const [1, 2, 3]</code>
Spread	<code>[...liste1, ...liste2]</code>
Héritage	<code>class Enfant extends Parent</code>
Mixin	<code>class A with B, C</code>
Asynchrone	<code>Future<T>, async, await</code>

Erreurs fréquentes à éviter



1. **Oublier required** dans un constructeur avec paramètres nommés → Erreur de compilation
2. **Assigner null à une variable non nullable** → Erreur de compilation
3. **Utiliser ! (bang operator) sans vérification** → Runtime error si null
4. **Oublier await** devant un Future → On manipule le Future, pas le résultat
5. **Modifier une const List** → Runtime error (const est immutable)

Check mentale avant de coder

- ☐ Mes variables sont-elles final quand elles ne changent pas ?
- ☐ Ai-je géré le cas null pour les types nullable ?
- ☐ Mes fonctions ont-elles des noms explicites ?
- ☐ Ai-je séparé mes classes par responsabilité ?
- ☐ Mes méthodes asynchrones ont-elles le mot-clé async ?

QUIZ DE VALIDATION – Module 1

Durée : 10 minutes – 8 questions

Question 1 (QCM)

Quelle est la différence entre final et const ?

- A. Aucune, ce sont des synonymes
- B. final est déterminé à la compilation, const à l'exécution
- C. const est déterminé à la compilation, final à l'exécution
- D. final permet la réassignation

Question 2 (QCM)

Quel opérateur permet d'appeler une méthode uniquement si l'objet n'est pas null ?

- A. !
- B. ?. • C. ??
- D. ??.

Question 3 (Ouvrte)

Écrivez la signature d'une fonction nommée calculerPrixTotal qui prend :

- un double prixUnitaire (requis)
- un int quantite (requis)
- un double? remise (optionnel, en pourcentage) Elle retourne un double. Utilisez des paramètres nommés.

Question 4 (QCM)

Quelle méthode permet de transformer chaque élément d'une List ?

- A. where
- B. map



- C. fold
- D. any

Question 5 (Ouverte)

Corrigez ce code qui contient une erreur de null safety :

```
String nom;  
print(nom.length);
```

Question 6 (QCM)

Dans une classe, `super.id` dans un constructeur signifie :

- A. On passe la valeur au constructeur de la classe parente
- B. On crée une variable locale nommée `super`
- C. On appelle une méthode statique
- D. C'est une erreur de syntaxe

Question 7 (Ouverte)

Écrivez une fonction asynchrone `recupererDonnees()` qui attend 1 seconde puis retourne la chaîne "Données chargées".

Question 8 (QCM – difficulté +) Quel est le résultat de ce code ?

```
var nombres = [1, 2, 3, 4, 5];  
var resultat = nombres.where((n) => n.isEven).map((n) => n * 2).fold(0, (a, b) => a + b);  
print(resultat);
```

- A. 20
- B. 12
- C. 30
- D. 6

Explication : `where` garde `[2, 4]`, `map` transforme en `[4, 8]`, `fold` additionne : $4 + 8 = 12$.

Barème indicatif

- 8/8 : Maîtrise parfaite, prêt pour le Module 2
- 6-7/8 : Acquis solides, petit point sur null safety ou asynchrone
- 4-5/8 : Révision nécessaire avant de passer au Module 2
- < 4/8 : Refaire les TP, le module 2 sera trop difficile

ANNEXE – Ressources et liens

- **DartPad** (en ligne, sans installation) : <https://dartpad.dev>
- **Documentation officielle Dart** : <https://dart.dev/language>
- **Cheat sheet Null Safety** : <https://dart.dev/null-safety/understanding-null-safety>
- **Exercices complémentaires** : <https://dart.dev/codelabs>